

M02 Student Practice Guide



M02 D3 概念词典与视觉映射 —— 零基础实践指南

导读：本指南是你连接设计思维与代码实现的桥梁。当你遇到满屏字母感到恐慌时，请随时回到“心智隐喻”寻找线索。不要害怕代码写错，报错与白屏是你最好的老师。



0. 课前准备：沙盒起飞与防患未然



目标与心智

Web 编程需要三个基础角色：**HTML(骨架图层)**、**CSS(化妆师)** 和 **JS(加工流水线)**。我们为你准备了一个免安装的在线沙盒：**CodePen**。为了防止以后使用 AI 辅写代码时遭遇版本冲突，我们需要给流水线安装一个稳定版本的 **D3 引擎**。



动作指令

1. 打开 [CodePen \(codepen.io\)](https://codepen.io) 并创建一个空白 Pen。
 2. 点击右上角 **Settings (设置)** -> 切换到 **JS 选项卡**。
 3. 🚫 **避坑警告**：不要直接搜索 “d3”（这会加载未经测试的最新版或未知版本）。
 4. 在底部的输入框中直接粘贴以下官方 v7 版本地址：`https://d3js.org/d3.v7.min.js`
 5. 点击 **Save & Close**。你的引擎已就绪！
-



实践一：Console 魔法三部曲（认识变量与函数）

目标与心智

用原生 JavaScript 控制网页。把**变量 (Variables)** 想象成贴了标签的“收纳盒”，把**函数 (Function)** 想象成“加工机器”。

动作指令

1.  **隔离污染**：不要按 F12 打开系统自带控制台，否则你的代码会把整个 CodePen 网页外壳染黑！
2. 请点击 CodePen 界面左下角自带的 **Console** 按钮，打开内置沙盒终端。
3. 依次复制并回车运行以下三段代码。

魔法代码

```
// 实验一：呼唤系统内置的巨型收纳盒 (document.body 代表整个页面)
document.body.style.background = "black";

// 实验二：制造你自己的收纳盒 (box) 与翻倍机器 (machine)
let box = 10;
let machine = d => d * 2;
console.log("单次加工结果：", machine(box));

// 实验三：让机器批量加工数据列表
let dataset = [5, 10, 15];
console.log("批量加工结果：", dataset.map(machine));
```

破坏与重塑 (Hack & Debug)

- **[探索]**：试着造一台名为 `triple` 的新机器，将数据 `d` 变成原来的 3 倍 (`d => d * 3`)，并批量加工 `dataset`。
- **[排错思路]**：如果你把 `let box = 10` 错写成 `let box = "10"` (加了引号变本为文本)，结果会如何？这就是数据类型的区别。

实践二：徒手画圆（认识 SVG 矢量透明图层）

目标与心智

SVG 是代码世界里的矢量画板。它就像永远保持图层面板打开的 **PSD**，里面的每一个形状（圆、矩形）都是独立的透明图层，可以随时单独修改。

动作指令

1. 切换到 CodePen 的 **HTML** 面板。
2. 粘贴以下代码，在预览区生成一个黑色的圆。

魔法代码

```
<svg width="500" height="50">
  <circle cx="25" cy="25" r="20" fill="black" />
</svg>
```

破坏与重塑 (Hack & Debug)

- **[探索]**：试着把 `cx` 改为 `100`，`r` 改为 `40`。看见圆怎么移动和变大了吗？
- **[排错指南]**：将鼠标移到右下角预览区，右键 -> 检查 (Inspect)。在浏览器弹出的 `Elements` 面板中，你会清晰地看到刚才写的 `<circle>` 图层。在这里你可以像在设计软件里一样，双击数值直接修改。这也是你未来检查 **D3** 生成结果的最强武器。

实践三：选择链与数据绑定 (Selection & Binding)

目标与心智

D3 的魔法是把数据贴在元素上。但这面临一个反直觉的步骤：如果要绑定 5 个数据，但目前桌上连一把椅子都没有，怎么办？**隐喻锚点**：`.selectAll()` 环顾全场找椅子，`.data()` 发放 5 个名牌，`.enter()` 发现缺椅子于是竖起 5 个虚拟占位牌，`.append()` 将占位牌变成真正的实体。

动作指令

1. 🚫 **避坑警告**：请先清空你在“实践二”中 HTML 面板里的所有代码！避免两张画布重叠引发“幽灵元素”。
2. 将以下代码粘贴到 CodePen 的 **JS 面板**中。

魔法代码

```
// 0. 防御性清理：每次重新运行前，清空旧画布
d3.selectAll("svg").remove();

// 1. 创建新画布
var svg = d3.select("body").append("svg").attr("width", 500).attr("height", 50);

// 2. 五步魔法咒语：绑定数据并生成缺失的段落
var dataset = [5, 10, 15, 20, 25];

d3.select("body")
  .selectAll("p")           // ① 环顾全场：找所有的段落（目前是空的）
  .data(dataset)           // ② 分发数据：5 个名牌
  .enter()                 // ③ 缺席宣告：发现没元素，竖起 5 个占位牌
  .append("p")             // ④ 实体化：把占位牌变成真实的 <p> 图层
  .text("数据已绑定！");    // ⑤ 填充内容
```

破坏与重塑 (Hack & Debug)

- **[排错] 白屏静默失败**：试着把第一行的 `d3.select("body")` 故意拼错成 `d3.select("boby")`。发生了什么？
 - **现象**：屏幕一片空白，而且控制台没有任何报错！
 - **原理**：D3 没找到元素时，会静默返回一个“空数组”，后续动作直接作废。
 - **救援兵**：遇到白屏，永远在代码中间加一句 `console.log(d3.select("你要找的元素"))`。如果打印出来长度是 0，说明抓空了。
- **[探索] 删掉 .enter()**：把 `.enter()` 这行删掉。5 个段落为什么全消失了？（因为你没有允许 D3 创建占位牌）。

📍 实践四：探针映射与空间排布 (Accessor Function)

🎯 目标与心智

我们生成了 5 个一模一样的元素，但这没意义。我们需要一根“取值探针 `function(d, i)`”，深入每个元素内部提取它独属的数据。 - `d` = Datum (数据本身，决定长得多大) - `i` = Index (排队序号 0,1,2,3..., 决定站在什么空间位置)

动作指令

1. 清空 **JS 面板** 的原有代码。
2. 粘贴以下进阶代码。注意观察 `i` 是如何推算 X 坐标 `cx` 的。

魔法代码

```
// 0. 防御性清理旧画布，并铺上一层浅灰底色方便观察边界
d3.selectAll("svg").remove();
d3.selectAll("p").remove();

var svg = d3.select("body")
    .append("svg")
    .attr("width", 500)
    .attr("height", 50)
    .style("background-color", "#f0f0f0");

var dataset = [5, 10, 15, 20, 25];

// 把原本的 <p> 换成 SVG 的 <circle>，并用数据控制坐标和半径
svg.selectAll("circle")
    .data(dataset)
    .enter()
    .append("circle")
    .attr("cx", function(d, i) {
        // 空间排布逻辑：序号 i (0,1,2,3,4) 乘以 50 的间距，再加上 25 像素的基础偏移量
        return (i * 50) + 25;
    })
    .attr("cy", 25)
    .attr("r", function(d) {
        // 视觉映射逻辑：半径直接使用数据值 d
        return d;
    })
    .attr("fill", function(d) {
        // [高亮条件判定] 如果数据 d 大于 15，涂成红色，否则涂成黑色
        return d > 15 ? "red" : "black";
    });
```

破坏与重塑 (Hack & Debug)

- **[探索] 坐标车祸：**试着把 `return (i * 50) + 25` 里的 `i` 删掉，直接返回 `50 + 25`。你会发现只剩下一个巨大的圆了！
 - **发生了什么：**5 个圆仍然存在（用右键 检查 Inspect 看看！），但因为坐标全都变成了固定值 75，它们叠在了一起。这就是**坐标失控**。
- **[探索] d 的失落：**试着把最后颜色的探针 `function(d)` 改成只写 `(d > 15 ? "red" : "black")`（去掉 `function` 包装）。
 - **现象：**报错或颜色失效。
 - **原理：**正如《教材》所说，“数据需要被函数的括号温暖地拥抱”。没有 `function` 作为管道，`d` 就不存在。